JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Interface TemporalAdjuster

All Known Subinterfaces:

ChronoLocalDate, ChronoLocalDateTime<D>, Era

All Known Implementing Classes:

DayOfWeek, HijrahDate, HijrahEra, Instant, IsoEra, JapaneseDate, JapaneseEra, LocalDate, LocalDateTime, LocalTime, MinguoDate, MinguoEra, Month, MonthDay, OffsetDateTime, OffsetTime, ThaiBuddhistDate, ThaiBuddhistEra, Year, YearMonth, ZoneOffset

Functional Interface:

This is a functional interface and can therefore be used as the assignment target for a lambda expression or method reference.

```
@FunctionalInterface
public interface TemporalAdjuster
```

Strategy for adjusting a temporal object.

Adjusters are a key tool for modifying temporal objects. They exist to externalize the process of adjustment, permitting different approaches, as per the strategy design pattern. Examples might be an adjuster that sets the date avoiding weekends, or one that sets the date to the last day of the month.

There are two equivalent ways of using a `TemporalAdjuster`. The first is to invoke the method on this interface directly. The second is to use `Temporal.with(TemporalAdjuster)`:

```
// these two lines are equivalent, but the second approach is recommended
temporal = thisAdjuster.adjustInto(temporal);
temporal = temporal.with(thisAdjuster);
```

It is recommended to use the second approach, `with(TemporalAdjuster)`, as it is a lot clearer to read in code.

The `TemporalAdjusters` class contains a standard set of adjusters, available as static methods. These include:

- finding the first or last day of the month
- finding the first day of next month
- finding the first or last day of the year
- finding the first day of next year
- finding the first or last day-of-week within a month, such as "first Wednesday in June"
- finding the next or previous day-of-week, such as "next Thursday"

Implementation Requirements:

This interface places no restrictions on the mutability of implementations, however immutability is strongly recommended.

Since:

1.8

See Also:

`TemporalAdjusters`

Method Summary

All Methods	Instance Methods	

Modifier and Type	Method	Description
`Temporal`	**`adjustInto`**`(Temporal temporal)`	Adjusts the specified temporal object.

Method Details

adjustInto

`Temporal adjustInto(Temporal temporal)`

Adjusts the specified temporal object.

This adjusts the specified temporal object using the logic encapsulated in the implementing class. Examples might be an adjuster that sets the date avoiding weekends, or one that sets the date to the last day of the month.

There are two equivalent ways of using this method. The first is to invoke this method directly. The second is to use `Temporal.with(TemporalAdjuster)`:

```
// these two lines are equivalent, but the second approach is recommended
temporal = thisAdjuster.adjustInto(temporal);
temporal = temporal.with(thisAdjuster);
```

It is recommended to use the second approach, `with(TemporalAdjuster)`, as it is a lot clearer to read in code.

Implementation Requirements:

The implementation must take the input object and adjust it. The implementation defines the logic of the adjustment and is responsible for documenting that logic. It may use any method on `Temporal` to query the temporal object and perform the adjustment. The returned object must have the same observable type as the input object

The input object must not be altered. Instead, an adjusted copy of the original must be returned. This provides equivalent, safe behavior for immutable and mutable temporal objects.

The input temporal object may be in a calendar system other than ISO. Implementations may choose to document compatibility with other calendar systems, or reject non-ISO temporal objects by `querying the chronology`.

This method may be called from multiple threads in parallel. It must be thread-safe when invoked.

Parameters:

`temporal` - the temporal object to adjust, not null

Returns:

an object of the same observable type with the adjustment made, not null

Throws:

`DateTimeException` - if unable to make the adjustment

`ArithmeticException` - if numeric overflow occurs

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.